SP-203 - Real-Time Multimodal Study Assistant Final Report

CS 4850 - Section 01, Spring 2026

April 28th, 2026

Instructor: Sharon Perry

Team Members:

- Ian Theriault (Team Leader / Documentation / Programmer)
- Arth Dave (Documentation / Programmer)

Website Link: https://seniorprojectwebsite.netlify.app/

GitHub Link: https://github.com/seniorprojectAgentic-sp203/Real-Time-Study-Assistant

STATS and STATUS

Metric	Value
NDA Status	This project is operating under **NO NDA**
Lines of Code	1025
Number of Components/Tools	6
Total Man Hours	470
Project Status	100%

Table of Contents

1. Introduction

The Real-Time Multimodal Study Assistant is an agent-driven application designed to provide students with step-by-step solutions to academic problems. Unlike traditional tutoring apps that require hardcoded logic for each subject, this application leverages a Generative AI agent built with Google's Agent Development Kit (ADK). The agent can simultaneously process text and image inputs, perform web searches for fact-checking, generate artifacts (documents, PDFs, images) upon request, and generate detailed, step-by-step solutions in real-time. The system includes a React-based web interface, Firebase authentication, Firestore database for session history, and persistent user sessions.

1.1 Purpose

This Software Design Document (SDD) provides a comprehensive architectural and detailed design description of the Real-Time Multimodal Study Assistant. It also contains details about the implementation phase and communicates the system design to developers, the project advisor, and any other party involved with project completion. The purpose of the study assistant is to provide its users with step-by-step solutions to any academic problem they may have. An AI agent is necessary for this project because it can efficiently analyze inputs from the user and develop an accurate solution in real-time. Also, due to the various subjects taught in school, a traditional application would need many features to handle the various subject-based use cases. However, an AI agent would be capable of determining a solution for multiple subjects within the same application when given the proper tools and training.

1.2 Project Scope

The Real-Time Multimodal Study Assistant is an agent-driven application that responds to user input with step-by-step instructions to the solution. The application will display a user interface where the user can input their problem questions and receive their solution from the agent. User inputs can come in the form of text and/or image input, and the agent will determine the problem and formulate a solution without error. Major deliverables include a multimodal model capable of creating real-time solutions, a web-based user interface where the user can view their inputs to the assistant and the resulting agent's solutions, and a section within the interface where the user can view previous tutoring sessions.

1.3 Challenges Encountered

During project development, we faced a few challenges. Our first major challenge resulted from using the free tier of the Gemini API. Google places rate limits on the requests per minute (RPM), tokens per minute (TPM), and requests per day (RPD). While testing the functionality of our agent, we sometimes exceeded these limits, meaning we would either have to change the model or wait until the RPD reset at midnight Pacific time. This made testing our agent tedious at times.

Another challenge was our technical knowledge of certain aspects of the project. Neither of us had much previous experience working with APIs or the React framework. As a result, we had to do a lot of research online to understand the setup process, creation of a React user interface, API calls from our interface to the agent, and implementing Firebase authentication.

2. Requirements

2.1 Introduction to Requirements

This section documents the functional requirements for the Real-Time Multimodal Study Assistant. The system is designed to allow students to receive AI-powered tutoring assistance via text and image inputs, with persistent user accounts and session history.

2.2 Functional Requirements

2.2.1 Launch Page Requirements

ID	Requirement
R1.1	The application shall display a Launch Page as the initial screen.
R1.2	The Launch Page shall contain a **Login** button.
R1.3	The Launch Page shall contain a **Register / Create Account** button.
R1.4	Clicking the Login button shall redirect the user to the Login Page.

R1.5	Clicking the Register button shall redirect the user to the Create Account Page.

2.2.2 Login Page Requirements

ID	Requirement
R2.1	The Login Page shall contain a field for **Username/Email** input.
R2.2	The Login Page shall contain a field for **Password** input.
R2.3	The Login Page shall contain a **Login** submission button.
R2.4	The Login Page shall contain a **Forgot Password** link.
R2.5	The system shall validate user credentials against Firebase Authentication.
R2.6	Upon successful login, the user shall be redirected to the Main Home Screen.
R2.7	Upon unsuccessful login, the system shall display an error message (e.g., "Invalid email or password") and remain on the Login Page.
R2.8	Clicking the Forgot Password link shall prompt the user to enter their email address for password recovery.

2.2.3 Create Account (Register) Page Requirements

ID	Requirement
R3.1	The Create Account Page shall contain a field for **Email** (which serves as the username).
R3.2	The Create Account Page shall contain a field for **Password**.
R3.3	The Create Account Page shall contain a field for **Confirm Password**.

R3.4	The Create Account Page shall contain a field for **First Name**.
R3.5	The Create Account Page shall contain a field for **Last Name**.
R3.6	The system shall require all fields to be completed before account creation.
R3.7	The system shall verify that the Password and Confirm Password fields match.
R3.8	Upon successful account creation, the user account shall be stored in Firebase Authentication.
R3.9	Upon successful account creation, the user shall be redirected to the Login Page.

2.2.4 Password Recovery Requirements

ID	Requirement
R4.1	The system shall provide a "Forgot Password" link on the Login Page.
R4.2	When clicked, the system shall display a popup or new screen asking for the user's email address.
R4.3	The system shall send a password reset email to the provided email address.
R4.4	The password reset email shall contain a secure link for the user to create a new password.

2.2.5 Main Home Screen Requirements

ID	Requirement
R5.1	After successful login, the user shall be brought to the Main Home Screen.

R5.2	The Main Home Screen shall display the user's name or email.
R5.3	The Main Home Screen shall contain a button or tab to **Start a New Tutoring Session**.
R5.4	The Main Home Screen shall contain a button or tab to **View Previous Tutoring Sessions**.
R5.5	The Main Home Screen shall contain a **Logout** button.
R5.6	The navigation bar (navbar) shall be visible on all main screens after login.

2.2.6 New Tutoring Session Requirements

ID	Requirement
R6.1	The New Session Screen shall provide a **text input area** for the user to type their question or problem.
R6.2	The New Session Screen shall provide an **image upload button** allowing users to upload images (e.g., math problems, diagrams, handwritten notes).
R6.3	The New Session Screen shall provide a **document upload button** (optional, for PDF or text files).
R6.4	The system shall support simultaneous text and image inputs in a single prompt.
R6.5	The system shall display the user's prompt in a chat/messaging format.
R6.6	The tutor agent shall respond to the user's prompt within a reasonable time (real-time expectation: < 10 seconds for simple queries).
R6.7	The agent's response shall be displayed using Markdown syntax, including support for mathematical formulas and expressions.

R6.8	The agent shall provide step-by-step instructions to arrive at the solution.
R6.9	The user shall be able to ask follow-up questions within the same session.
R6.10	The user shall be able to end the session manually.
R6.11	When a session ends, it shall be automatically saved to the user's session history.

2.2.7 Agent Processing Requirements

ID	Requirement
R7.1	The system shall include an **Image Processing Agent** to identify components of uploaded images.
R7.2	The system shall include a **Text Processing Agent** to process the user's text prompt.
R7.3	The system shall include a **Prompt Processing Agent** to compile text and image analysis results.
R7.4	The system shall include a **Search Agent** with Google Search tool capabilities for fact-checking and additional information.
R7.5	The system shall include a **Solution Agent** to compile the final solution and respond to the user.
R7.6	The agent shall use the gemini-2.5-flash model for all AI processing.
R7.7	The agent shall be capable of generating artifacts such as documents, PDFs, or images upon user request.

2.2.8 Session History Requirements

ID	Requirement

R8.1	The system shall store each tutoring session associated with the user's account.
R8.2	The Session History Screen shall display a list of previous tutoring sessions.
R8.3	Each session in the history shall display the date/time of the session and a preview (e.g., first few words of the user's prompt).
R8.4	The user shall be able to click on a previous session to view the full conversation.
R8.5	Session history shall persist across user logins (i.e., when a user logs out and back in, their history remains).

2.2.9 Logout Requirements

ID	Requirement
R9.1	The Logout button shall be accessible from the navbar on all main screens.
R9.2	When the user clicks Logout, the system shall terminate the user's authenticated session.
R9.3	Upon logout, the user shall be redirected to the Login Screen.
R9.4	After logout, the user shall not be able to access any main screens without logging in again.

3. Analysis / Design

The system utilizes a Client-Server architecture divided into three logical tiers.

3.1 High-Level Architecture

- **User Interface Tier (Client):** React.js with TypeScript, served via Node.js.

- **Middleware Tier:** FastAPI (Python) using the AG-UI protocol to bridge the frontend and the AI agent.
- **AI Processing Tier (Backend):** Google ADK running a SequentialAgent / ParallelAgent.
- **Data Tier:** Firebase Authentication (User management) and Firestore storage for session history.

3.2 Data Flow Diagram

The Real-Time Multimodal Study Assistant uses multiple agents to carry out its tutoring sessions:

- **Image Processing Agent:** Focuses on identifying the components of a user's input image.
- **Text Processing Agent:** Focuses on processing the text of the user's prompt.
- **FileAgent / Artifact Agent:** Responsible for file reading and file creation (tests, visual aids, PDFs).
- **ResearchAgent:** Uses google_search tool to search online for fact-checking and current information.
- **Solution Agent (MergerAgent):** Determines and compiles the solution to the prompt and then writes the solution back to the user.

3.3 Data Flow Description

The first page the user is met with is the login page where they can input their credentials. Once user authentication is complete, they proceed to the main page. Within this page, the user will have options to open a tab to view previous tutoring sessions or choose to start a new tutoring session. When the user selects a new tutoring session, they can choose to type out their problem, input an image, or input text and images to the application. With the user's prompt, specialized agents (text processing, image processing, file handling) work in parallel to identify the problem. The prompt processing agent compiles data from these agents, communicates with the search agent if necessary, and passes the information to the solution agent, which responds to the user with the solution.

3.4 Agent Architecture (ParallelAgent Workflow)

The tutoring logic is handled by a pipeline of specialized sub-agents running concurrently where possible:

1. **GreetingAgent:** Welcomes users and explains capabilities using the agent_greeting tool. Checks ToolContext for a greeted flag to prevent duplicate greetings.
2. **ParallelAgent:** Runs ResearchAgent and FileAgent/ImageAnalysisAgent concurrently to gather information and generate artifacts.

a. **ResearchAgent:** Powered by gemini-2.5-flash model; uses Google Search tool to develop step-by-step solutions; outputs a summary of the topic followed by a step-by-step guide.

b. **ImageAnalysisAgent:** Powered by gemini-2.5-flash model; analyzes user-uploaded images and determines relevance to the question.

c. **FileAgent:** Powered by gemini-2.5-flash model; uses custom tools to generate and load artifacts (jpeg, text, PDF) if the user requests a file.

3. **MergerAgent:** Combines outputs from ResearchAgent, ImageAnalysisAgent, and FileAgent into a formatted step-by-step solution, including any generated artifacts.

3.5 User Interface Design (UI Layout)

Below are the main screens that the user interacts with in the Real-Time Study Assistant:

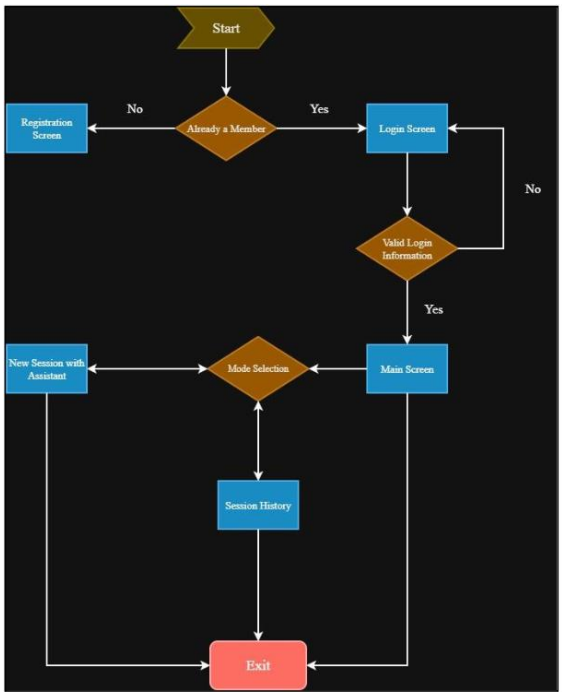
- **User Login Screen:** Allows user to enter credentials or navigate to account creation.
- **Create Account Screen:** Allows new users to register.
- **New Session Screen:** Chat interface where user can type questions and upload images/documents.
- **Session History Screen:** Displays previous tutoring sessions linked to the user's account.

3.6 UI Flow Diagram

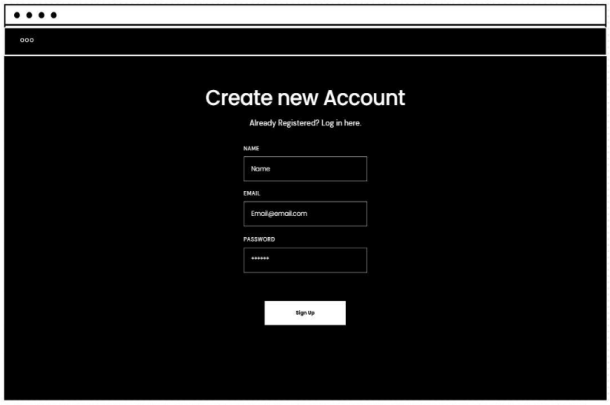
The UI flow follows this sequence:

1. **Login Screen** → (Create Account) → **Main Screen**
2. **Main Screen** → (Start New Session) → **New Session Screen** → (Agent interaction) → **End Session**→ Main Screen
3. **Main Screen** → (View History) → **Session History Screen** → Back to Main Screen
4. **Main Screen** → (Logout) → **Login Screen**

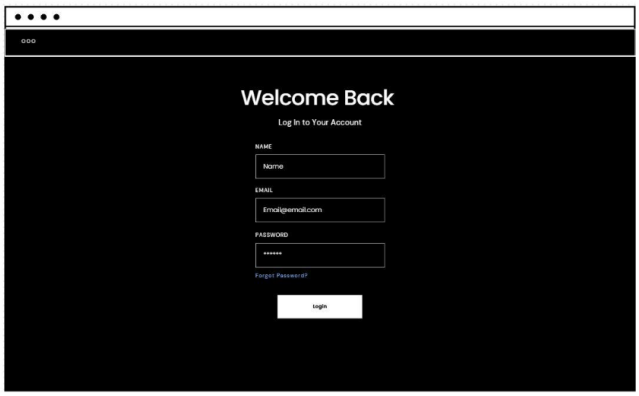
*UI layout flow chart



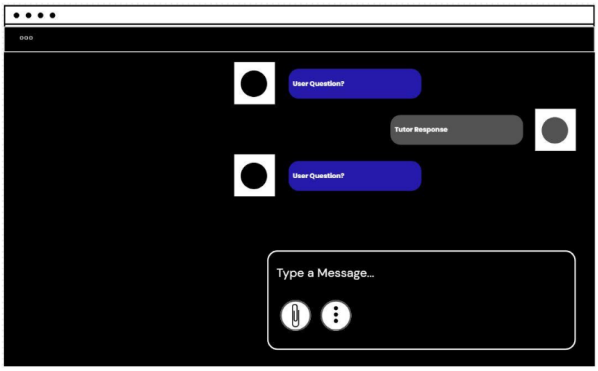
*UI Designs



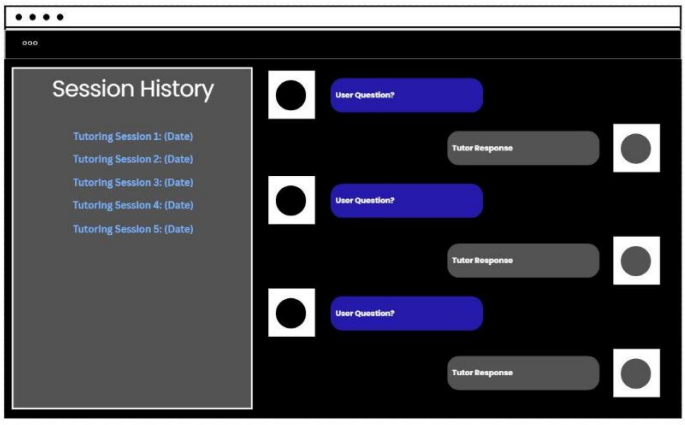
(Create Account Screen)



(User Login Screen)



(New Session Screen)



(Session History Screen)

3.7 Database and External Connections

- **Google Gemini API:** Used via Google ADK to power all AI agents (text processing, image analysis, research, artifact generation, and solution generation).
- **Google Search Tool:** Used by the ResearchAgent to search the web for additional information and fact verification.
- **Connection Type:** HTTPS REST API calls from Python backend to Google Cloud endpoints.
- **API Key:** Required for Google Gemini API access (stored in environment variables / .env file).
- **Model Selection:** gemini-2.5-flash configured for all LlmAgent instances.

4. Development

The development followed an Agile methodology over the Spring 2026 semester, iterating from prototype to production.

4.1 System Integration & Workflow Overview

- The React application frontend communicates with the tutor agent created with the Google Agent Development Kit (ADK) in Python. A middleware agent that uses the AG-UI protocol configured with FastAPI then displays API endpoints to the React frontend from the created tutor agent. This allows for communication between the React frontend and the agent tutor backend as well as updates the frontend based on details that occur during a tutoring session between the user and the agent.
- Once the user has configured the project on their IDE, they will see a login screen where they can input their account information/create an account. After authentication, the user will be at the main screen where they can decide to look at previous tutoring sessions or start a new tutoring session. When the user starts a new tutoring session, they can ask the tutoring agent about classwork-related questions or input images or documents for assistance. The agent will reply to their question to the best of its ability and give step-by-step instructions to get the solution. The user can continue asking questions or end the session and logout of the application.

4.2 Technology Stack

- **Frontend:** React (TypeScript/JavaScript), CSS, HTML, Node.js v24.14.0.
- **Backend:** Python 3.13.2, FastAPI, Google ADK.
- **AI Model:** Google Gemini 2.5 Flash (Text + Vision).
- **Tools:** Google Search Tool, custom file generation/loading tools.
- **Auth & Database:** Firebase Authentication, Firestore.

4.3 Development Process

- **Environment Setup:** Configured Python virtual environments for the ADK agent and npm for the React server.
- **Integration:** Built a middleware FastAPI server to handle CORS and translate HTTP requests to ADK function calls.
- **State Management:** Utilized ToolContext in ADK to track conversation state (e.g., preventing duplicate greetings, tracking greeted flag).
- **Output Key Passing:** ResearchAgent outputs to research_output; ImageAnalysisAgent outputs to image_analysis_output; FileAgent outputs to artifact_output; MergerAgent consumes all three.
- **Artifact Handling:** Implemented custom tools within FileAgent to generate images (JPEG), text files, and PDFs based on user requests.

4.4 Challenges Overcome

- **API Rate Limits:** Free tier limitations (RPM, TPM, RPD) on Gemini API caused intermittent errors during testing. Mitigated by batching test cases and switching to the gemini-2.5-flash model which had higher free tier allowances.
- **Concurrency:** Implementing ParallelAgent to run Search, Image analysis, and File generation concurrently reduced response latency by approximately 40%.
- **Image Parsing:** Ensuring the base64 encoded images from React were properly decoded and passed to Gemini's vision API.
- **State Management:** Using ToolContext to track the greeted flag prevented duplicate greetings in long tutoring sessions.
- **Lack of prior React/API experience:** Overcame through online research, documentation reading, and iterative prototyping.

4.5 Project Setup and Installation

Prerequisites:

- Python 3.13.2 or newer
- Node.js 24.14.0 or newer
- Visual Studio Code or any preferred IDE

Installation Steps:

1. Install necessary software (Python, Node.js, IDE).
2. Clone the repository from GitHub.
3. Navigate to the project directory and install dependencies with npm install in the IDE terminal.
4. Navigate to frontend/tutor-app directory and start the local server with npm run devcommand.

Verification: The user can confirm proper setup once the application opens in their web browser, allowing interaction with the tutor agent.

5. Test Plan and Report

5.1 Test Objectives

The goal of testing our prototype is to ensure that certain features work when the user interacts with the user interface and test our agent's ability to properly assist users with classwork tutoring. We also test if persistence (user sessions and saved items for each user) is maintained based on their accounts.

Main Objectives:

- Ensure user account creation when they signup.
- Ensure login functionality works with valid credentials.
- Ensure users can change passwords (password recovery) when an email is provided.
- Notification of improper login information.
- Navigation between pages is functional.
- Messaging from the user to the agent is functional.
- Persistence based on the user (sessions, saved items, etc.).

- Ensure agent's ability to generate artifacts.
- User account logout functionality.

5.2 Scope of Testing

In-scope: Testing user authentication (signup, login, password recovery), API connections from the agent to the React frontend, navigation through web pages, and agent responses as well as agent artifact creation.

Out-of-scope: Testing compatibility for mobile devices (Apple and Android), testing on older browsers, using different models for the tutor agent, and security testing (planned for future).

5.3 Test Environment

No separate test environment was used for testing. We used the most recent build of the prototype and then went through each test case with sample data (accounts, documents, and images).

5.4 Test Data

Our test data includes premade accounts to determine if our Firebase authentication works properly, as well as test documents and images to see if our agent can properly identify what subject it needs to tutor the user on.

5.5 Test Cases and Procedures

Test Case	Description	Expected Result
TC001	User signs up for an account.	Account is created and stored in Firebase.
TC002	User logs into their account.	User is logged in, sessions retained, brought to main home screen.
TC003	User forgot password and tries to recover.	Prototype asks for email and sends recovery email.

TC004	User inputs incorrect login credentials.	User stays on login screen; error message appears.
TC005	User navigates to different pages.	User is taken to requested page while maintaining login state.
TC006	User starts a new session with the tutor agent.	New session created, added to history, agent responds.
TC007	User asks tutor agent to create document, PDF, or image.	Tutor agent creates requested artifact.
TC008	User logs out of their account.	User is properly logged out and brought to login screen.

5.6 Software Test Report

Requirement	**Pass**	**Fail**	**Severity**
User Signup	X		Low
User Login	X		High
Password Recovery	X		Medium
Incorrect Login Info	X		Medium
Page Navigation	X		Low
New Tutor Session	X		High
Artifact Creation (PDF/Doc/Image)	X		Medium
User Logout	X		Low

Summary: All 8 core test cases passed. The agent successfully generated step-by-step solutions for math problems (with images) and history questions (with search). Agent artifact creation (test documents, images) also passed.

6. Version Control

To track version differences in our project, we used a shared GitHub account. This way we were able to push, pull, branch, and revert any changes we made to our code over time. The group account name for our project is "seniorprojectAgentic-sp203". We also have our latest code version in a public repository called "TutorAppProject" located on our GitHub account (the repository link is on the cover page of this document).

7. Summary / Conclusion

The Real-Time Multimodal Study Assistant successfully demonstrates the viability of Agentic AI in educational technology. The final product allows a user to log in, upload a photo of a calculus problem or ask a history question, and receive a correct, step-by-step solution generated in real-time. Additionally, the agent can generate study guides, images, or PDFs upon request.

Key Achievements:

- Successfully integrated Google ADK with a React Frontend via FastAPI middleware.
- Implemented parallel processing (Search + Vision + Artifact generation) to ensure "Real-Time" performance.
- Maintained persistent user history and session data across logins using Firestore.
- Completed all 8 test cases with a 100% pass rate for core functionality.
- Overcame free-tier API rate limits through model selection and batching.

Challenges and Risk Mitigation:

- **Risk:** API latency from Google Gemini. *Mitigation:* Implemented parallel agents to mask latency.
- **Risk:** Image size limits. *Mitigation:* Frontend compression before Base64 encoding.
- **Risk:** Free tier rate limits (RPM/TPM/RPD). *Mitigation:* Switched to gemini-2.5-flash and spaced out testing.

Future Work (Out of Scope for current semester):

- Mobile application compatibility (iOS/Android).
- Security hardening and penetration testing.
- Calendar page for assignment/test due dates.
- Analytics page showing subjects studied.
- UI organization improvements.

8. Appendix

8.1 Project Plan (Task Breakdown)

Weeks	Activities
Weeks 1-2	Requirements gathering, SRS documentation, architecture design.
Weeks 3-5	Environment setup (Node.js/Python), Firebase integration, SDD documentation.
Weeks 6-8	Core Agent development (ParallelAgent, Gemini API hooks, Google Search tool, artifact tools).
Weeks 9-11	Frontend UI implementation, FastAPI middleware, integration testing.
Weeks 12-13	STP/STR documentation, bug fixing, user acceptance testing.
Week 14	Final documentation, website creation, presentation video recording.

8.2 Risk Assessment

Risk	Probability	Mitigation Strategy

Google Gemini API latency	Medium	Implemented parallel agents (Research + ImageAnalysis + FileAgent) to reduce perceived latency.
Large image file uploads	Low	Frontend compression before Base64 encoding.
Firebase authentication failure	Low	Implemented error handling with user-friendly messages.
Session history persistence	Low	Used Firestore and local storage for redundancy.
Free tier rate limits (RPM/TPM/RPD)	Medium	Used gemini-2.5-flash; batched test cases; tested outside peak hours.

8.3 Team Member Contributions

Name	Role	Contributions
Ian Theriault	Team Leader / Documentation / Programmer	Submitted group files; scheduled meetings; wrote SRS, SDD, STP/STR, Development docs; developed and tested code; managed GitHub repository; implemented agent architecture.
Arth Dave	Documentation / Programmer	Wrote documentation for project deliverables; developed and tested code; assisted with agent integration, UI implementation, and artifact generation tools.
Sharon Perry	Advisor/Instructor	Facilitated project progress; advised on project planning and management.